



Ministry of Education
Ovidius University of Constanța
Faculty of Mathematics and Computer Science
Speciality Cyber Security and Machine Learning

Digital Proof Abstraction and Investigation

Work of disertation

Scientific coordinators:

Assoc. Prof. Marius Iulian Mihăilescu

Assoc. Prof. Punchianu Crenguța Mădălina

Graduate:

Parfion Vlăduț

Constanța

2023

Table Of Contents

Table Of Contents	i
List of Figures	1
1 Abstract	2
2 Introduction	3
2.1 Background	3
2.1.1 Topic Introduction	3
2.1.2 Current State of the Field	4
2.2 Problem statement	5
2.3 Objective	5
2.4 Scope and limitations	6

3	Literature Review	8
3.1	The Concept of Digital Proof Abstraction	8
3.2	Existing Approaches to Digital Proof Investigation	9
3.3	Clustering Algorithms in Data Analysis	10
3.4	Existing Applications and Tools for Digital Proof Investigation	12
3.5	Gaps in the Current Research	13
4	Methodology	14
4.1	Selection of Tools: Python and PyQt5	14
4.2	The K-means Algorithm: Explanation and Usage	15
4.3	Design and Implementation of the Application	16
4.4	User experience	28
5	Testing and Results	31
6	Discussion	32
6.1	Potential real-world applications	32
6.2	SWOT analasys	33
6.2.1	Strengths	33
6.2.2	Weaknesses	33

6.2.3	Opportunities	34
6.2.4	Threats	34
7	Conclusion and Future Work	35
7.1	Conclusion	35
7.2	Future Work	35
	Bibliography	36

List of Figures

4.1	Main interface	17
4.2	File selection	18
4.3	Pan left click	20
4.4	Multiple point comment	21
4.5	Single comment point	22
4.6	Scrollwheel zoom	23
4.7	Move point	24
4.8	K-means1	25
4.9	K-means2	26
4.10	pdf1	27
4.11	pdf2	29

Chapter 1

Abstract

English:

In this dissertation, the challenge of abstracting meaningful proofs from the ever-growing volume of digital data is addressed. An application was developed using Python and PyQt5 to assist in this task, utilizing the K-means algorithm for grouping and coloring data points, thereby facilitating pattern recognition. The application provides a user-friendly 2D interface, allowing investigators to navigate through data, annotate individual or group points, and generate investigative reports. After extensive testing, the application demonstrated potential to streamline the process of digital proof investigation, enhancing efficiency in data analysis. With its implications extending to fields such as cybersecurity, law enforcement, and legal dispute resolution, this research and application contribute significantly to ongoing efforts in digital forensics.

Romanian:

În această disertație, se abordează provocarea de a abstractiza dovezi semnificative din volumul în creștere continuă de date digitale. A fost dezvoltată o aplicație folosind Python și PyQt5 pentru a asista în această sarcină, utilizând algoritmul K-means pentru gruparea și colorarea punctelor de date, facilitând astfel recunoașterea modelului. Aplicația oferă o interfață 2D ușor de utilizat, permițând investigatorilor să navigheze prin date, să adnoteze puncte individuale sau de grup și să genereze rapoarte de investigație. După teste extensive, aplicația a demonstrat potențialul de a simplifica procesul de investigare a dovezilor digitale, îmbunătățind eficiența în analiza datelor. Având implicații care se extind la domenii precum securitatea cibernetică, aplicarea legii și rezolvarea litigiilor legale, această cercetare și aplicație contribuie semnificativ la eforturile continue în domeniul forensicii digitale.

Chapter 2

Introduction

2.1 Background

2.1.1 Topic Introduction

Digitalization has drastically transformed how we live, work, and communicate. As the volume of digital data grows exponentially, so too does the potential for this data to be used as evidence in a wide range of contexts, from criminal investigations to corporate disputes. This has given rise to a field known as 'digital forensics', which involves the collection, analysis, and interpretation of digital evidence.

A key part of digital forensics is 'digital proof abstraction and investigation', the process by which digital evidence is identified, extracted, and interpreted. As data becomes increasingly complex and voluminous, abstracting meaningful proofs from this mass of information is becoming both more challenging and more critical.

In recent years, sophisticated algorithms and applications have been developed to aid in digital proof abstraction. They are designed to sift through large amounts of data, identify patterns, and extract relevant information that can serve as 'digital proof'. These technologies have proven invaluable in many scenarios, including detecting cybercrimes, identifying fraud, and resolving legal disputes.

However, digital proof abstraction and investigation is not without its challenges. These include ensuring the accuracy and integrity of digital proofs, protecting privacy, and managing the sheer volume and complexity of digital data. Moreover, interpreting digital proofs

requires considerable expertise, as the context is often as important as the evidence itself.

This study delves into the intricacies of digital proof abstraction and investigation, exploring the challenges, the technologies, and the implications for society. It also presents a novel application developed to aid in the abstraction and investigation of digital proofs, demonstrating its effectiveness through a series of tests and analyses.

2.1.2 Current State of the Field

Digital proof abstraction and investigation currently sits at the intersection of numerous disciplines including computer science, law enforcement, and digital forensics. As the generation of digital data has exploded in the past decade, digital proof abstraction has grown in relevance and complexity.

Existing methods in the field primarily focus on the identification, extraction, and analysis of digital evidence from a variety of sources like system logs, databases, and electronic communication. Various tools and applications have been developed to help in these tasks, leveraging algorithms and advanced data analytics to search through large volumes of data and extract relevant proofs.

One key development in recent years has been the use of machine learning and artificial intelligence in digital proof abstraction. These technologies are capable of processing vast amounts of data at high speeds, identifying patterns and anomalies, and even learning and adapting over time to improve their effectiveness.

However, despite these advances, the field still faces significant challenges. Ensuring the accuracy and integrity of digital proofs remains a complex problem, especially given the ease with which digital data can be manipulated. Additionally, issues of privacy and data protection have become increasingly pressing as more and more personal information becomes digitized.

The field struggles with the growing volume and complexity of digital data, which compounds the task of identifying relevant proofs, especially with new data types and sources emerging continually.

Therefore, while significant progress has been made in digital proof abstraction and investigation, there is still much work to be done. The field continues to evolve in response to technological advances, legal developments, and societal changes.

2.2 Problem statement

Despite the advances in the field of digital proof abstraction and investigation, there remains a significant challenge in processing the sheer volume and complexity of digital data. Traditional methods can be slow and inefficient, often requiring substantial human involvement and expertise. Moreover, these methods may struggle to effectively identify and interpret complex patterns within the data.

Another key issue is the usability of existing tools and applications. While some are powerful, they often require a high degree of technical knowledge to use effectively, which can limit their utility for investigators who are not experts in data analysis.

Furthermore, there is a need for tools that not only facilitate the extraction of digital proofs, but also allow investigators to easily navigate, annotate, and document their findings. These features are critical for supporting a comprehensive and systematic investigation process.

This research seeks to address these problems through the development and testing of a new application that leverages clustering algorithms to enhance digital proof abstraction, provides a user-friendly interface for investigation, and supports the generation of detailed investigation reports.

2.3 Objective

1. To design and develop an application for digital proof abstraction and investigation - This involves choosing appropriate tools (Python, PyQt5) and algorithms (K-means), and designing a user-friendly interface.
2. To enhance the process of digital proof abstraction using the K-means algorithm - This involves implementing the algorithm in your application and using it to group and color data points.
3. To create a 2D navigational interface for easier investigation of data points - The objective is to create an interface that allows investigators to interact with the data in a more intuitive and efficient way.

4. To enable annotation and documentation of individual or grouped data points within the application - This will provide investigators with a tool for keeping track of their findings and thoughts during the investigation process.

5. To facilitate the generation of detailed investigation reports - This functionality is aimed at making the investigator's job easier by automatically generating a report from their annotations and selected data points.

6. To evaluate the effectiveness and efficiency of the application in digital proof investigation - Through a series of tests, assess whether the application meets the goals of enhancing digital proof abstraction and investigation.

7. To provide recommendations for future improvements to the application and the field of digital proof abstraction and investigation - This could be based on the findings from the application tests, feedback from users, or new developments in the field.

2.4 Scope and limitations

Scope:

1. Algorithm and Application Development: The research includes the development of an application that employs the K-means clustering algorithm for digital proof abstraction. This will involve software development in Python and PyQt5.

2. Testing of Application: The research will include testing of the developed application. Tests will be designed to evaluate the effectiveness and efficiency of the application in abstracting and investigating digital proofs.

3. Documentation and Reporting: The study covers the design and implementation of features within the application that allow for user annotation and report generation.

4. Comparison and Analysis: The research will analyze the performance of the developed application and compare it to existing methods or applications for digital proof abstraction and investigation.

Limitations:

1. **Data Privacy and Security:** While the application will be designed with data security in mind, this research will not delve deeply into advanced cybersecurity or privacy issues related to the handling of digital proofs.
2. **Algorithm Selection:** The study is limited to the use of the K-means algorithm for clustering. Other clustering algorithms might be effective but are outside the scope of this research.
3. **Limited Testing:** Testing of the application will be based on available datasets and use cases. Results may vary with different types or volumes of data.
4. **User Expertise:** The effectiveness of the application is in part dependent on the expertise of the user, which is a variable that the study cannot control.

Chapter 3

Literature Review

3.1 The Concept of Digital Proof Abstraction

The concept of digital proof abstraction involves the process of identifying, extracting, and interpreting pieces of digital information that can serve as evidence in a given context. This practice is a crucial aspect of digital forensics, which deals with the collection and analysis of digital evidence.

As more and more of our activities move online or are facilitated by digital devices, a wealth of digital data is continuously being generated. This data can range from system logs and databases, to emails and social media posts, and even digital images or video files. Any of these data types can contain valuable information that might serve as a proof, i.e., evidence, in investigations ranging from criminal cases to corporate disputes.

However, given the sheer volume and complexity of digital data, simply finding the relevant proofs can be a daunting task. This is where digital proof abstraction comes into play. It involves using techniques, algorithms, and tools to sift through large amounts of data, identify the pieces of information that might be relevant for a particular case, and then extract these pieces for further analysis.

This process often involves a degree of "abstraction". This means that rather than dealing with raw data, investigators often work with a more simplified or general representation of the data that still retains its essential aspects. For instance, a large set of log files might be abstracted into a timeline of key events, or a database might be abstracted into a set of relevant records.

However, the process of digital proof abstraction isn't just about finding and extracting data. It's also about interpreting the data and understanding its significance in the context of a particular case. This often requires a deep understanding of both the technical aspects of the data and the specific circumstances of the case.

Overall, digital proof abstraction is a complex and multi-faceted process that forms a crucial part of digital forensics and plays a key role in modern investigations. As technology continues to evolve and digital data continues to grow, the importance and complexity of this field are likely to increase.

3.2 Existing Approaches to Digital Proof Investigation

Digital proof investigation has been a rapidly evolving field. Over the years, various approaches and methodologies have been developed and adopted, which can be broadly classified into manual and automated methods.

Manual Approaches:

Initially, manual methods of investigation were more prevalent. Investigators would manually search through files, emails, logs, and other data sources to find relevant information. This could involve techniques such as keyword searches, timeline analysis, or link analysis. While these methods can be effective, they are time-consuming and require a high degree of expertise.

Automated Approaches:

With the growth of digital data, automated tools and techniques have become increasingly important in digital proof investigation. These can significantly speed up the investigation process and handle larger and more complex datasets.

1. Data Recovery Tools: These are used to recover lost or deleted data from digital devices. Examples include Recuva, TestDisk, and PhotoRec.

2. Forensic Imaging Tools: These tools create an exact copy of a digital device's storage, preserving the data in its original state for investigation. Examples include FTK Imager and Guymager.

3. **Analysis Tools:** These tools help investigators search for and analyze digital evidence. They may offer features like keyword searching, hash matching, metadata analysis, or timeline analysis. Examples include Autopsy and EnCase.

4. **Network Forensics Tools:** These are used to collect and analyze network traffic, which can provide valuable information about network-based attacks or unauthorized network activity. Examples include Wireshark and NetworkMiner.

Advanced Approaches:

More recently, advanced techniques involving machine learning and artificial intelligence have been adopted to further enhance digital proof investigation.

1. **Machine Learning (ML):** ML algorithms can be trained to recognize patterns in data, which can be particularly useful in identifying suspicious or abnormal behavior.
2. **Natural Language Processing (NLP):** NLP can be used to analyze text-based data, such as emails or social media posts, to extract relevant information or identify suspicious content.
3. **Deep Learning:** Deep learning, a subset of ML, can be used for more complex tasks like image or video analysis, which can be useful in investigations involving multimedia data.

While these approaches have significantly improved the speed and effectiveness of digital proof investigation, they also pose new challenges in terms of ensuring the accuracy and integrity of digital proofs, protecting privacy, and managing the complexity of advanced analytical tools. As the field continues to evolve, further research and development are needed to address these challenges.

3.3 Clustering Algorithms in Data Analysis

Clustering algorithms are powerful tools in data analysis and have found a variety of applications in fields such as machine learning, image recognition, market research, and, as is relevant to your research, digital proof abstraction and investigation. These algorithms aim to segregate a dataset into subsets or "clusters" such that data points within the same cluster are more similar to each other than to those in other clusters.

K-Means Algorithm

In the context of your dissertation, the K-means algorithm is of particular importance. K-means is a centroid-based or partitioning algorithm that separates data into K groups of similarity. The algorithm iteratively assigns each data point to one of K groups based on the features that are provided. Data points are clustered based on feature similarity, which can be any metric that quantifies the difference between data points (often Euclidean distance).

K-means is relatively simple to implement and efficient to run on large datasets, making it one of the most widely used clustering algorithms. It can help reveal structure within data that may not be immediately apparent, making it useful for exploratory data analysis.

Applications in Digital Proof Abstraction and Investigation

Clustering algorithms, like K-means, can be very useful in the context of digital proof abstraction and investigation. They can help investigators to identify patterns or outliers in large datasets, which might be indicative of relevant evidence.

For example, consider a case where an investigator is looking at log data from a server to find unusual activity. If the log data is transformed into a suitable format (e.g., numerical features indicating login times, IP addresses, etc.), a clustering algorithm can be used to group similar log entries together. This could make it easier for the investigator to spot groups of log entries that stand out from the rest, which could be a sign of unusual or suspicious activity.

Clustering can also be useful in situations where an investigator has to deal with unstructured data, like text. For example, text data can be transformed into a numerical format (e.g., using TF-IDF or word embeddings) and then clustered to identify groups of similar text entries.

While clustering can provide valuable insights and assist in the investigation process, it's important to note that it's largely an exploratory tool. The results can depend heavily on how the data is preprocessed and what parameters are chosen for the algorithm. Therefore, the interpretation of the results often requires both domain knowledge and a good understanding of the algorithm and data.

3.4 Existing Applications and Tools for Digital Proof Investigation

Numerous tools and applications have been developed to facilitate the process of digital proof investigation. These range from comprehensive digital forensics suites that provide a wide range of capabilities, to specialized tools that focus on specific aspects of the investigation process.

1. **Autopsy:** This is a comprehensive digital forensics platform that provides a wide range of capabilities, including file and data recovery, timeline analysis, and keyword searching. It also supports a variety of plugins, allowing it to be adapted to specific investigative needs.
2. **EnCase Forensic:** Developed by OpenText, EnCase Forensic is a widely used digital investigation tool. It provides numerous features, including disk imaging and preservation, in-depth data analysis, and comprehensive reporting.
3. **FTK (Forensic Toolkit):** Another popular suite, FTK offers a range of features such as data carving, email analysis, and password recovery. It is known for its ability to handle large volumes of data.
4. **Wireshark:** This is a network protocol analyzer often used in digital investigations involving network traffic. It allows investigators to capture and interactively browse the traffic running on a computer network.
5. **Volatility:** A memory forensics framework that allows investigators to extract digital artifacts from volatile memory (RAM) dumps.
6. **OSForensics:** This tool allows investigators to identify suspicious files and activity with hash matching, drive signature comparisons, e-mails, memory, and binary data.

In addition to these tools, machine learning-based methods have begun to be incorporated into digital forensics. For example:

1. **Cognitive Forensic Analyst:** Developed by IBM, this tool uses artificial intelligence to assist in investigations by analyzing data and suggesting areas that investigators should focus on.
2. **X-Ways Forensics:** A powerful application for data recovery and digital forensics that also incorporates data clustering for easier data analysis.

While these tools provide a wide range of capabilities, they also require a certain level of

technical expertise to use effectively. Moreover, there is still a need for tools that can efficiently handle the increasing volume and complexity of digital data.

3.5 Gaps in the Current Research

Identifying gaps in current research is crucial for establishing the significance of your work and demonstrating how it contributes to the field. For the area of digital proof abstraction and investigation, several gaps might be identified:

1. **Scalability:** While many tools and techniques have been developed for digital proof abstraction, there's still a significant challenge in dealing with the ever-increasing volume and complexity of digital data. Existing methods may struggle to scale effectively, leading to increased processing times and decreased accuracy.
2. **Integration of Machine Learning Techniques:** Despite the potential of machine learning algorithms to improve digital proof abstraction, their application in this field remains relatively limited. More research is needed to understand how these techniques can be effectively integrated into digital forensics tools and workflows.
3. **Usability:** Many existing tools for digital proof abstraction require a high degree of technical expertise to use effectively. There's a need for more user-friendly applications that can be used by investigators without a background in data analysis or computer science.
4. **Interactive Analysis:** Current tools often lack sufficient capabilities for interactive analysis. For example, there might be limited functionality for navigating through a data set, annotating individual data points, or generating comprehensive reports.
5. **Validation of Results:** Ensuring the accuracy and integrity of digital proofs is a complex problem, given the ease with which digital data can be manipulated. More research is needed on methods for validating the results of digital proof abstraction and ensuring that they can stand up to scrutiny in a legal context.
6. **Data Privacy:** As digital proof abstraction often involves dealing with sensitive data, issues of privacy and data protection are a major concern. There's a need for more research on how to effectively protect privacy while still allowing for effective investigation.

Chapter 4

Methodology

4.1 Selection of Tools: Python and PyQt5

When embarking on the development of the application for digital proof abstraction and investigation, the choice of programming tools was a critical consideration. Python, coupled with the PyQt5 framework, emerged as an ideal combination for several compelling reasons.

Python's robustness and versatility set it apart as one of the preferred languages for developing complex applications. Yet, beyond its capacity to manage intricate tasks, it's Python's inherent simplicity that made it an attractive choice. Its clean syntax and user-friendly design make translating thought into code almost intuitive. This simplicity does not come at the expense of power; Python is equipped with an extensive set of libraries, making it possible to handle everything from intricate mathematical computations to machine learning processes – a feature critical in managing the complexities of digital proof abstraction.

But the decision to use Python extended beyond the language itself. The strength of a language is often reflected in the community that supports it, and Python's community is vast and active. This vibrant ecosystem provides a wealth of resources and problem-solving capacities that is invaluable during development.

However, developing an application that is not only functional but also user-friendly was paramount. Enter PyQt5, a set of Python bindings for the Qt libraries, renowned for their powerful and flexible UI and software development tools. PyQt5 marries the robustness of Qt with the simplicity of Python, enabling the creation of a sophisticated yet intuitive user interface.

Using PyQt5 allowed us to develop an application that could handle complex tasks like data clustering and visualization while remaining accessible and engaging for the user. Its compatibility with Python meant we could seamlessly integrate backend functionality with frontend design, crafting an application that is cohesive, efficient, and easy to navigate.

Ultimately, the fusion of Python and PyQt5 provided us with a canvas on which we could design and implement an application tailored to the needs of digital proof investigation, balancing computational power with user experience, and complexity with accessibility.

4.2 The K-means Algorithm: Explanation and Usage

The K-means algorithm is a popular centroid-based clustering algorithm used in machine learning and data mining. At its core, K-means seeks to partition a dataset into K distinct, non-overlapping clusters, where each data point belongs to the cluster with the nearest mean.

The algorithm follows these general steps:

1. The number of clusters, K, is initialized. The choice of K is critical and depends on the specific dataset and the goal of the clustering.
2. K "centroids" are randomly initialized. These are not necessarily actual data points but rather K randomly placed points in the feature space.
3. Each data point in the dataset is assigned to the nearest centroid, and K clusters are formed.
4. For each of the K clusters, the centroid is updated to be the mean of all data points in the cluster.
5. Steps 3 and 4 are repeated iteratively until the algorithm converges. Convergence is typically defined as the point where the centroids no longer move significantly or the cluster assignments no longer change.

Despite its simplicity, the K-means algorithm is highly effective and scalable, making it suitable for a wide range of clustering tasks.

Usage in the Project:

In the context of digital proof investigation, the K-means algorithm offers an efficient way to group related data points. Given the high-dimensional nature of digital data, visually identifying patterns can be challenging. The K-means algorithm helps by grouping similar data points together based on their features.

In your application, the K-means algorithm is used to group and color the points in the 2D space. This allows for more effective visualization and interpretation of the data. By moving in this colored 2D space, the investigator can more easily identify clusters of similar data points, which can lead to more efficient discovery of digital proofs.

Furthermore, the iterative refinement of clusters in the K-means algorithm offers a dynamic and refined view of the data. The investigator can, therefore, derive insights more efficiently, even from large and complex datasets.

It's important to note that while the K-means algorithm can significantly aid in digital proof investigation, the choice of K and the interpretation of the results require careful consideration and domain expertise. This underscores the need for the combination of automated methods and human expertise in digital proof investigation

4.3 Design and Implementation of the Application

Application architecture:

The application is composed of multiple classes. I use class MainApp to organize and manage the graphic interface. Minor functionalities are implemented in the same class like loading a csv file but most of the other functionalities are used in separate classes that handle actions and process the data.

The input file is a csv which we use as a static data structure but for the most part I use a dynamic data structure for handling the process of grouping dots with the k-means algorithm and storing comments for dots and group of dots separately.

While the app is centered around k-means algorithm most of the work has gone into the graphic interface due to the difficulty of finding documentation and adding features without breaking the application.

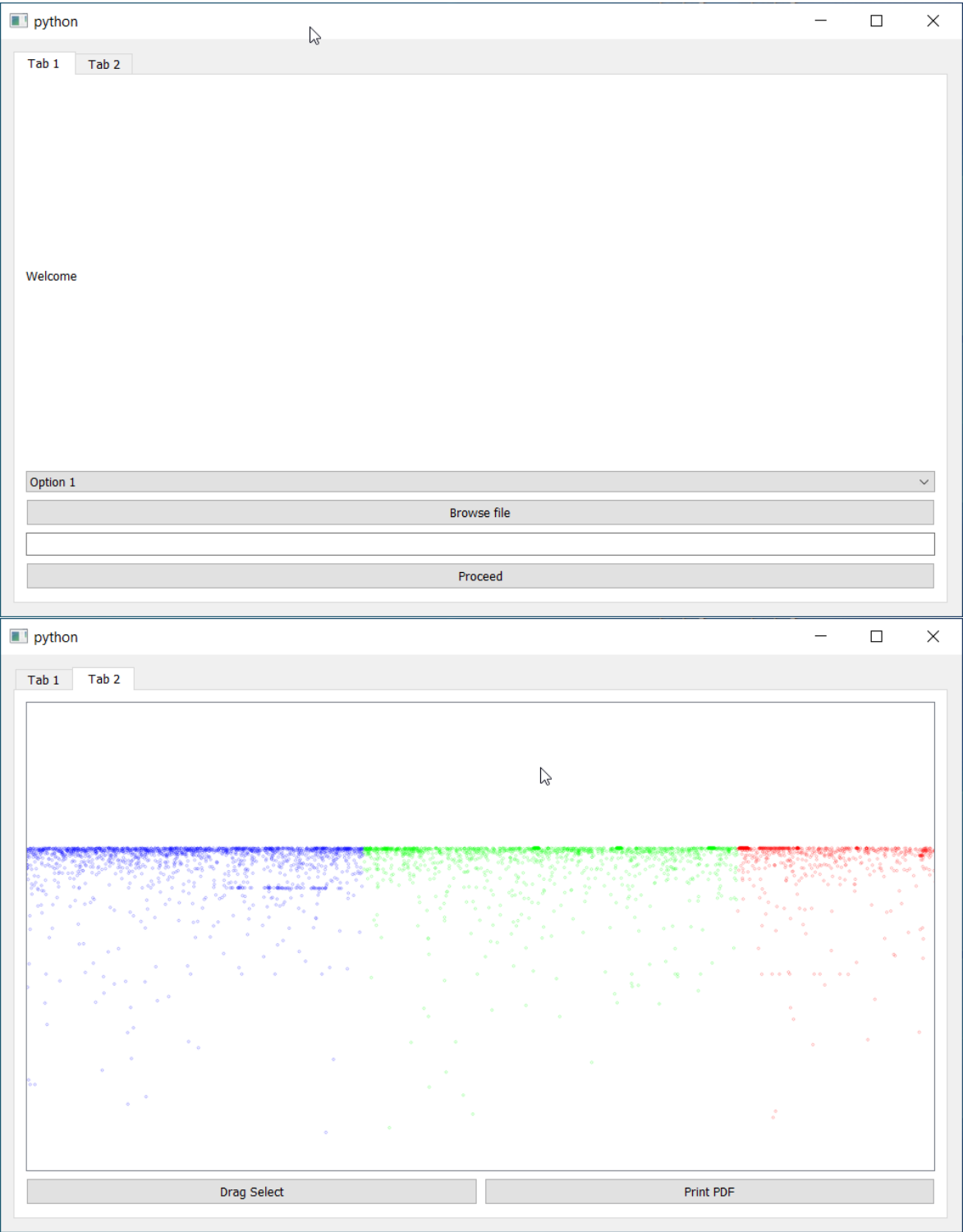


Figure 4.1: Main interface

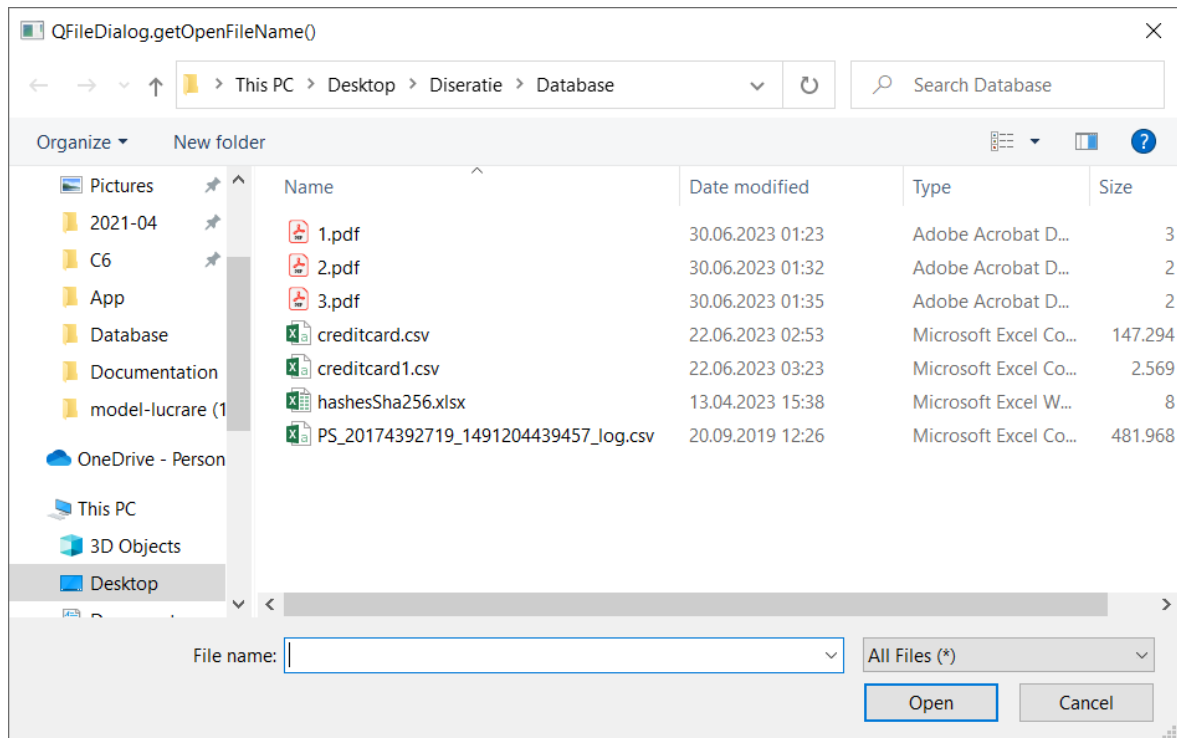


Figure 4.2: File selection

User Interface Design:

When starting the application the user is greeted by a window with two tabs. The first tab is used to prepare the workspace and the second one is used to display and work with the data point of the csv without modifying it.

In the first tab 4.1 the user is greeted by a label that greets the user, a combo box for selecting the algorithm that we want to use, a button named "browse file" that we use to select the csv file and upon pressing it we are greeted by a user interface 4.2 which we can use to select the file we want, a textbox which I can manually introduce the path to the file which is also updated when we select the file using the "browse file" button and "proceed" button which we use to process the list of dots and fill the two dimensional space with those.

On the second tab the user will be able to see all the data points distributed on a virtual two dimensional space followed by two buttons underneath which are used to toggle drag-select on or off and a print PDF button which opens another window for adding additional details.

In the new window we can fill the two additional textbox items and save the result of the investigation in a PDF.

Implementation of the K-means Algorithm:

'Initialization():'

The initialization method for the class takes an array of data points as an input and stores it in the class instance.

Processing (process):

The processing method is where the K-means algorithm is implemented. It takes the number of clusters k as an input.

Internal Methods:

The processing method defines several internal methods:

centroidnp(arr): This method calculates the centroid of a given array of data points.

dist(A,c2): This method calculates the Euclidean distance of each data point to each centroid. It then assigns each data point to the cluster of the nearest centroid.

arrange(A): This method arranges the data points in the array based on their cluster assignments.

centroid(c1): This method computes the centroid of each cluster.

K-means Iterative Process:

In the main body of the process method, the iterative process of the K-means algorithm

takes place: 4.8 4.9

1. Initially, it assigns a random cluster to each data point.
2. Then it enters a loop that continues until the cluster assignments do not change:
It saves the current state of the data points to a1. It arranges the data points based on their current cluster assignments and calculates the new centroids. It reassigns each data point to the cluster of the nearest centroid and saves the new state to a2. It checks if the state of the data points has changed. If not, it breaks out of the loop.
3. Once the loop is completed, the method returns the final state of the data points, with each data point assigned to a cluster.

The KMeans class provides a flexible and easy-to-use implementation of the K-means algorithm in Python. This can be used to classify data points into clusters, aiding in the analysis and investigation of digital proofs.

Implementation of Data Navigation and Annotation Features:

There are multiple features added in order to help the user all being in for the second tab:

1. Move in the 2d space with left click

The "Move in the 2D space with left click" functionality is implemented using the

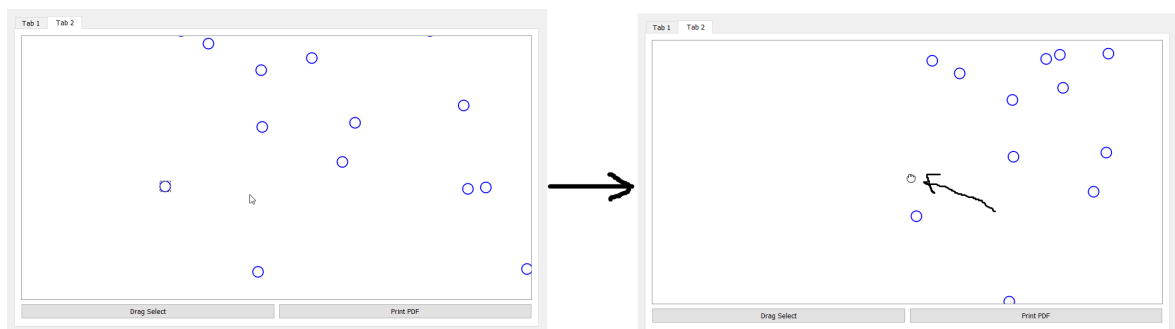


Figure 4.3: Pan left click

panning feature in the MyGraphicsView class.

How it works:

The `mousePressEvent` method checks if the right mouse button is clicked. If it is, the panning variable is set to `True`, and the last mouse position (`last-mouse-pos`) is updated with the current mouse position. The cursor is also changed to a "closed hand" cursor to indicate the panning mode.

When the mouse moves (`mouseMoveEvent`), if panning is enabled (`panning` is `True`), the method calculates the difference between the current mouse position and the last mouse position (`delta`). This difference is then added to the current scroll bar values, effectively moving the view.

The `mouseReleaseEvent` method checks if the right mouse button is released. If it is, the panning variable is set to `False`, and the cursor is changed back to the default "arrow" cursor, effectively ending the panning mode.

So, to pan (or move in the 2D space), the user needs to press and hold the right mouse button, move the mouse, and then release the button. The view will move according to the mouse movement.

2. Drag-select comment

The "Drag-select comment" functionality is implemented primarily in the `MyGraphicsView` class, with some assistance from the `DotItem` class and the `CommentDialog` class. How it works: The `MyGraphicsView` class has a `QPushButton` named `toggle-`

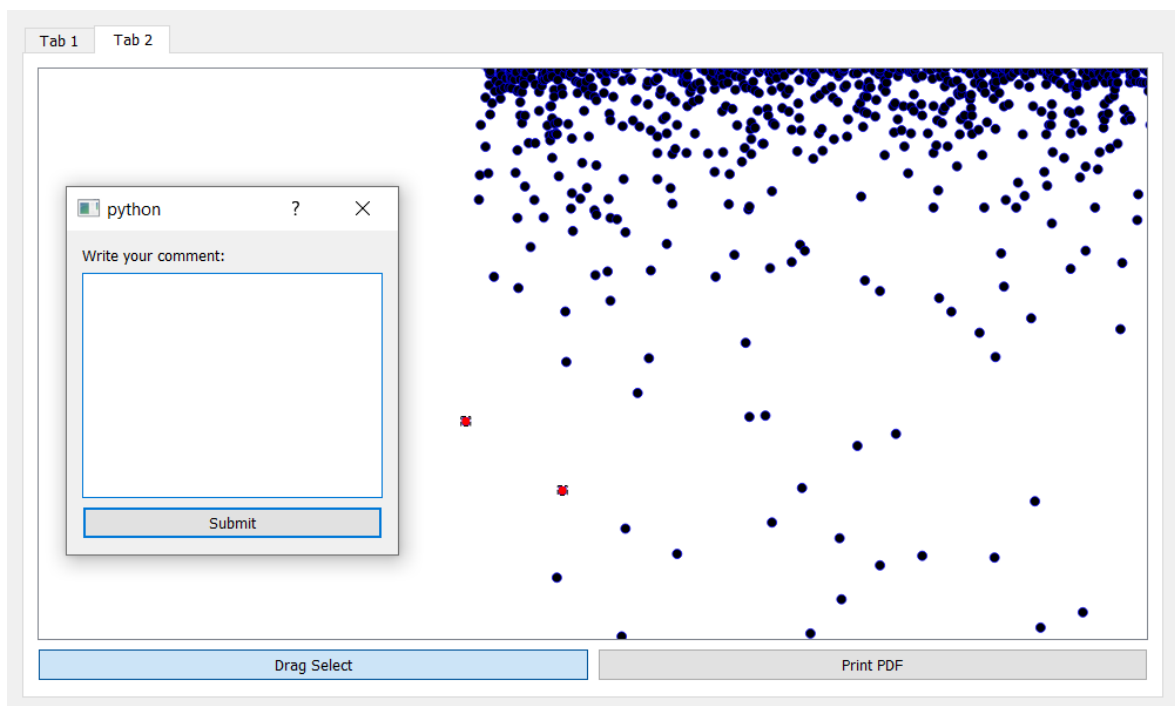


Figure 4.4: Multiple point comment

button which, when toggled, sets the drag-select-mode to `True` or `False`.

When a mouse press event (`mousePressEvent`) is detected and the drag-select-mode is `True`, it saves the starting position of the drag in `drag-start-pos` and creates a new rectangle for the selection area (`selection-rect-item`).

During a mouse move event (`mouseMoveEvent`), if the `drag-select-mode` is `True`, it updates the rectangle to match the area between the starting position of the drag and the current mouse position.

When the mouse is released (`mouseReleaseEvent`), if the `drag-select-mode` is `True`, it creates a polygon from the selection rectangle and retrieves all `DotItem` objects that intersect with this polygon (i.e., all dots within the selection area). It then sets these dots as selected and changes their color to red, while all other dots are deselected and their color is set to black.

It then checks if more than one dot was selected. If so, it creates a `CommentDialog` and passes the identifiers (coordinates) of the selected dots to it.

In the `CommentDialog`, the user can write a comment for the group of selected dots. When the "Submit" button is clicked, it appends the comment to the global `global-comments` list and closes the dialog. The comment, along with the identifiers of the selected dots, can then be retrieved using the `get-comments` method.

So, to drag-select dots and add a comment for them, the user needs to enable the "Drag Select" mode by clicking the "Drag Select" button, drag the mouse to select multiple dots, release the mouse button, and then write a comment in the dialog that appears.

3. Double click comment

The "Double click comment" functionality is implemented in the `DotItem` class. Here's how it works: The `DotItem` class, which represents a data point (dot) in the graphical

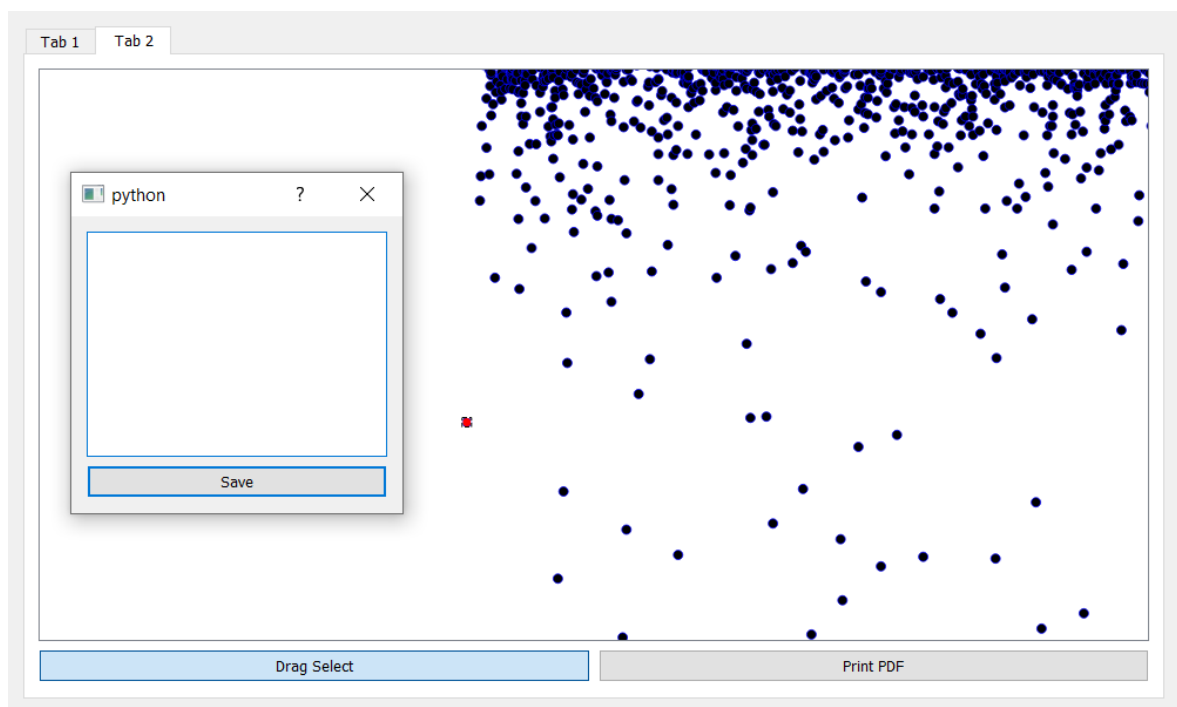


Figure 4.5: Single comment point

view, has a `mouseDoubleClickEvent` method. This method is called whenever the user double-clicks on a dot.

In this method, it creates a new QDialog and sets up a QVBoxLayout layout for it.

It creates a QTextEdit widget with the current comment of the dot (which is initially an empty string) and adds it to the layout. This widget allows the user to write or edit the comment.

It also creates a "Save" button and connects its clicked signal to a lambda function. This function calls the -save-comment method with the current text of the QTextEdit widget and the dialog as arguments.

The -save-comment method updates the comment of the dot (self.-comment) with the new text and closes the dialog.

After setting up the dialog, it calls the exec method to show the dialog and start its event loop. This will block the program until the user closes the dialog.

So, to add or edit a comment for a dot, the user needs to double-click on the dot, write the comment in the dialog that appears, and click the "Save" button. The comment can then be retrieved using the get-comment-and-coordinates method.

4. Scroll zoom

The "Scroll zoom" functionality is implemented in the MyGraphicsView class. How it works:

The MyGraphicsView class has a wheelEvent method. This method is called when-

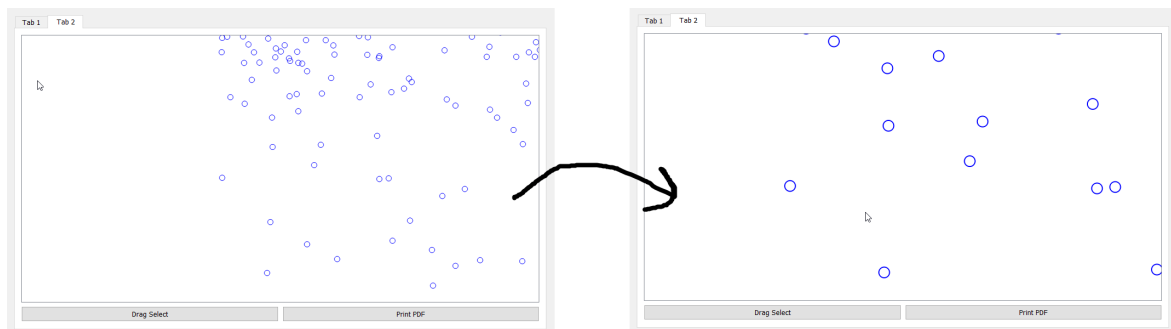


Figure 4.6: Scrollwheel zoom

ever the user scrolls the mouse wheel while the mouse cursor is over the MyGraphicsView widget.

In this method, it first defines two factors: zoom-in-factor and zoom-out-factor. These factors determine how much the view should be zoomed in or out for each tick of the mouse wheel.

It then checks the angleDelta().y() of the event. This is a measure of how much the wheel was scrolled, and its sign indicates the direction of the scroll. If it's positive, the user scrolled up, so it sets zoom-factor to zoom-in-factor to zoom in. If it's negative, the user scrolled down, so it sets zoom-factor to zoom-out-factor to zoom out.

Finally, it calls the scale method with the zoom-factor as both the x and y factors. This scales the view by the zoom-factor, effectively zooming in or out.

So, to zoom in or out, the user needs to scroll the mouse wheel up or down, respectively, while the mouse cursor is over the MyGraphicsView widget. The view will zoom towards or away from the point under the cursor.

5. Move point

The "Move point" functionality is implemented in the DotItem class, which represents a data point (dot) in the graphical view. This class inherits from QGraphicsEllipseItem, which in turn inherits from QGraphicsItem. The QGraphicsItem class includes a built-

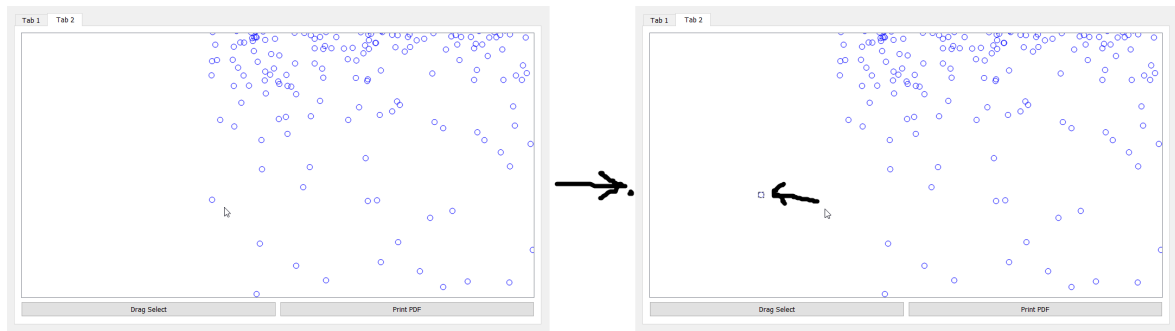


Figure 4.7: Move point

in mechanism for handling item movement. When an item is clicked and dragged, the item's `mouseMoveEvent` method is called repeatedly as the mouse moves, and the item's position is automatically updated to follow the mouse.

In the `DotItem` class, this built-in mechanism is enabled by the line `self.setFlag(QGraphicsEllipseItem.ItemIsMovable)`. This line sets a flag that tells Qt to enable the built-in movement handling for this item.

However, the `DotItem` class also includes its own `mouseMoveEvent` method, which is called whenever the mouse moves over a `DotItem`. In this method, it checks if the left mouse button is pressed and if the view is in drag select mode. If both conditions are true, it ignores the event, which prevents the dot from being moved when the user is trying to select multiple dots by dragging the mouse. If these conditions are not true, it calls `super().mouseMoveEvent(event)`, which allows the event to be handled by the superclass's method (i.e., the built-in movement handling).

So, to move a dot, the user needs to click and hold the left mouse button on the dot, move the mouse to the desired location, and then release the button. However, this only works when the view is not in drag select mode. If the view is in drag select mode, the dot will not move, and instead, multiple dots can be selected by dragging the mouse.

Implementation of Report Generation:

The "report generation" functionality is implemented in several classes: `Generator`, `CommentWindow`, and `MainApp`. How it works:

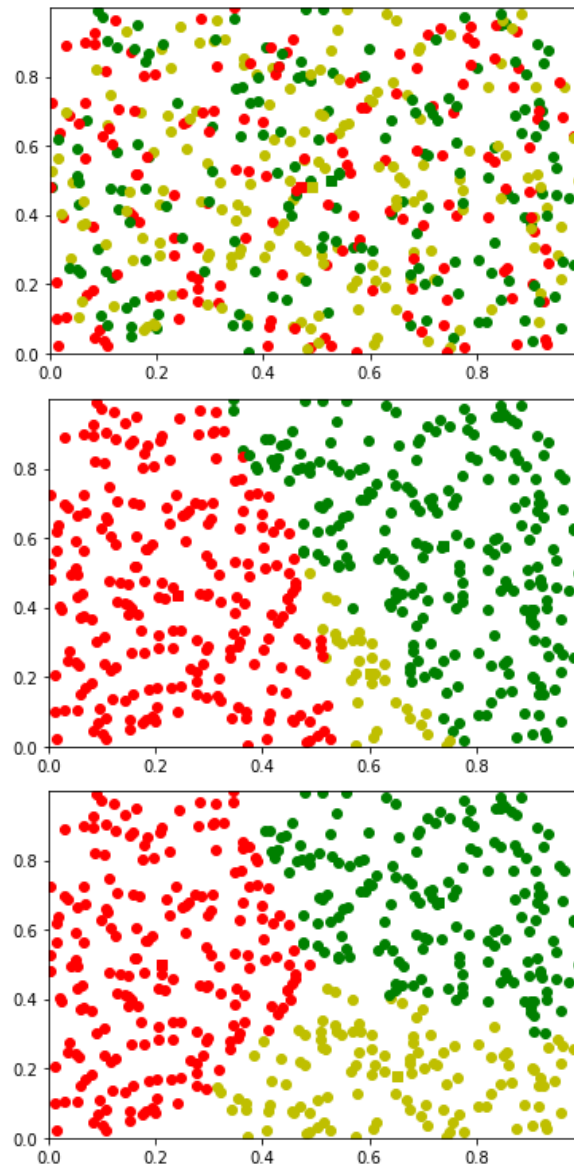


Figure 4.8: K-means1

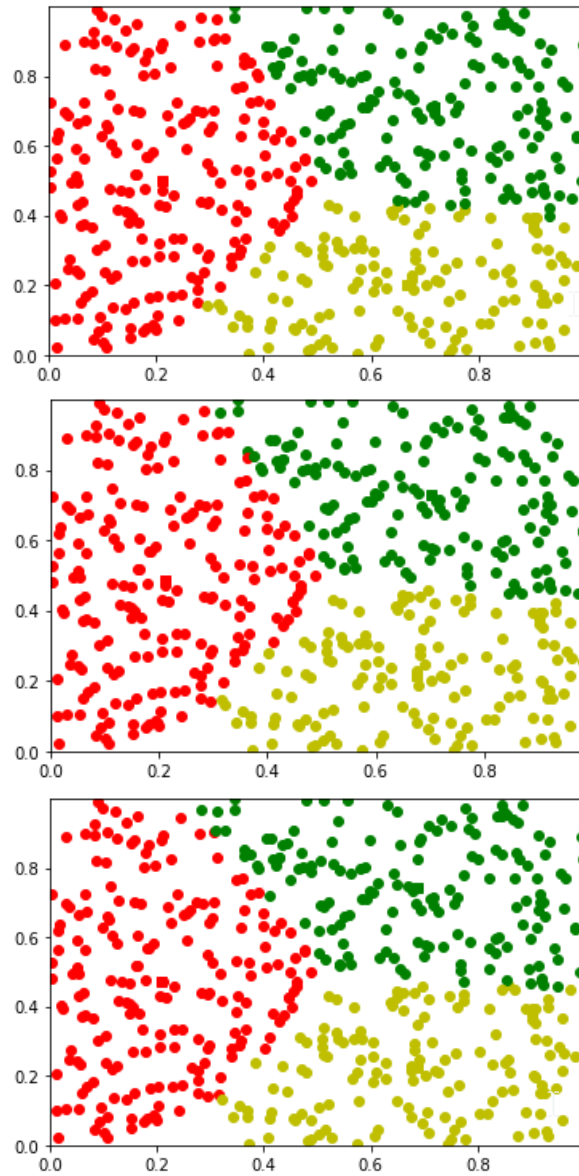
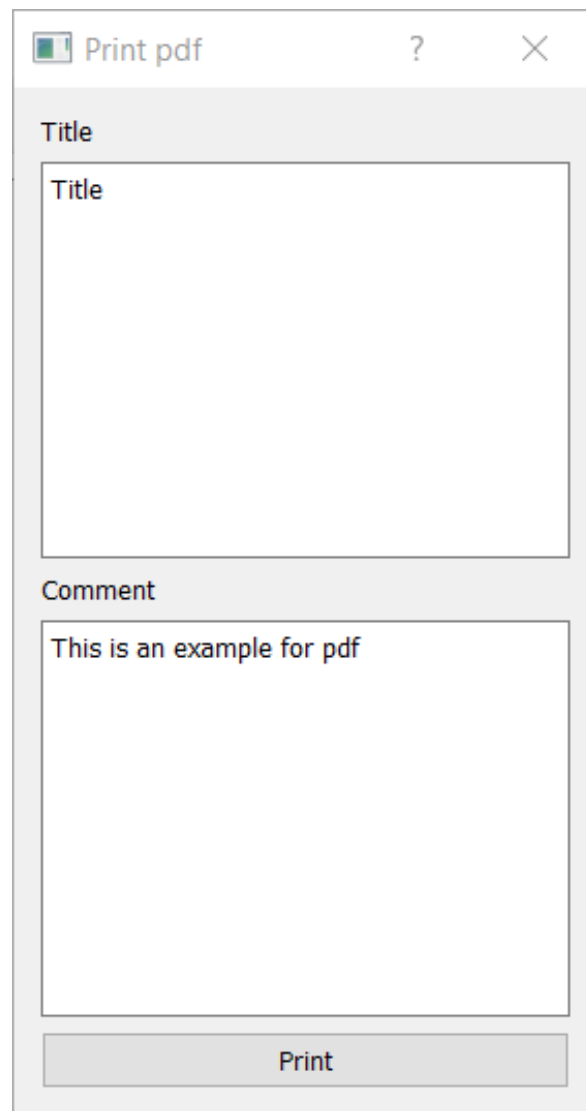


Figure 4.9: K-means2



Print pdf

Title

Title

Comment

This is an example for pdf

Print

Figure 4.10: pdf1

In the MainApp class, a "Print PDF" button (button2) is created. When this button is clicked, it triggers the on-button2-clicked method. This method collects comments from group selections and individual dots, merges them together, and then creates a new CommentWindow ??, passing the merged comments to it.

The CommentWindow class is a dialog that allows the user to enter a title and main comment for the report. It also has a "Print" button. When this button is clicked, it triggers the start-pdf-generation method. This method first collects the title and main comment from the text boxes and puts them, along with the additional comments passed to the CommentWindow, into a dictionary. It then opens a file dialog asking the user where to save the PDF file. If a file path is chosen, it creates a new Generator instance, passing the data dictionary and file path to it, and starts the Generator in a new thread.

The Generator class is a QRunnable that generates the PDF report. It has a run method which is called when the Generator is started. In this method, it creates a new PDF file at the specified file path and writes the title, main comment, and additional comments into the file. If an error occurs during this process, it emits an error signal with the error message. If the process completes successfully, it emits a file-saved-as signal with the file path. ??

4.4 User experience

As it may be observed most of the interactions with the user interface are done with the use of the mouse. The design choice has been done for the following reasons:

1. Intuitiveness:

Using a mouse is a highly intuitive process. The on-screen cursor movements correspond to the physical movements of the mouse, which makes the interaction feel natural. This lowers the learning curve for users, allowing them to navigate and interact with applications more easily.

2. Precision:

The mouse allows for precise control over the cursor's position. This is especially useful when selecting specific items or regions, such as individual or groups of data points in your application. Precise control enhances user interaction and provides a greater degree of freedom in how users interact with the interface.

3. Quick Access:

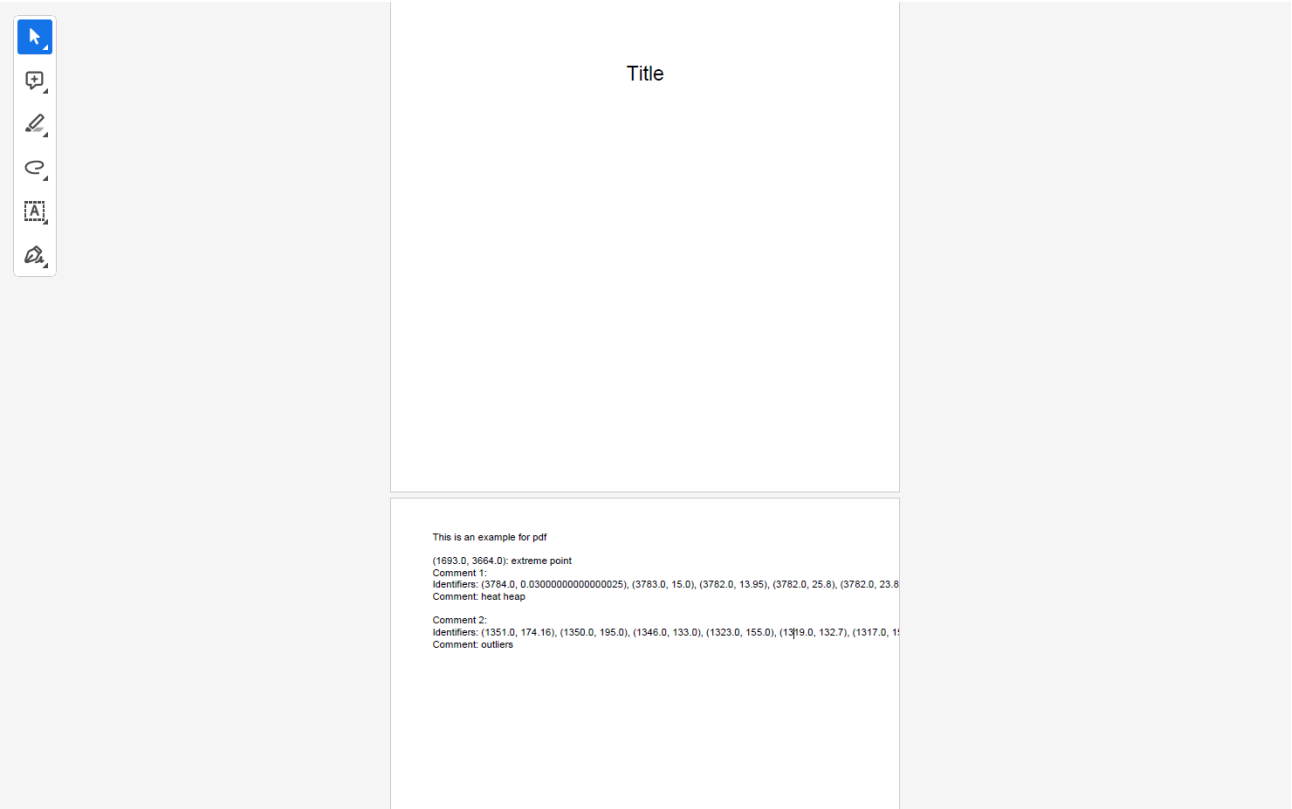


Figure 4.11: pdf2

The mouse allows users to directly access the features of an application. Users can quickly navigate through different parts of the interface, click on options, and drag-and-drop elements. This significantly reduces the time needed to perform actions compared to keyboard-only interfaces.

4. Versatility:

The mouse allows for a range of different inputs, including clicking, right-clicking, double-clicking, scrolling, and dragging. This versatility makes it possible to build a rich, interactive user interface with a variety of different controls and options.

5. Ergonomics:

Mice are designed to be comfortable for long-term use. They are shaped to fit the hand and require minimal effort to move, which reduces the risk of strain or discomfort during long sessions of use.

Chapter 5

Testing and Results

How is the application being tested:

1. Iterative Development and Testing:

During the development of the application i have used a method in adopting and implementing features. By developing a new feature and then immediately testing it, I ensured each function worked correctly before moving on to the next.

2. Unit and Functional Testing:

Each time I added a new function to your application, I conducted what is essentially a combination of unit testing (testing individual components) and functional testing (testing functionality of the application). I ensured that the function itself worked correctly and that it did not break any existing functionality of the application.

3. Problem-Solving and Debugging:

When a function is not working properly I am using mostly print statements. The strategy I am using to find the problem and solve it is by following the flow , inserting print in key zones and trying to understand the problem.

4. Results of Iterative Testing:

While there were no major "before and after" results to discuss the process exposed several coding flaws which resulted in minor roadblocks while programming. One example would be trying to recover the comment over a group of dots while finding out that the dots stored inside the class would end up being deleted the moment the window is being closed which is when I push the submit button which also closes the window deleting all the data in the process.

5. Lessons Learned:

While I used Iterative Development in order to make the application , it proved to be the right choice due to the complexity of the project but also due to novelty of the topic.

Chapter 6

Discussion

6.1 Potential real-world applications

1. Identifying Patterns:

The K-means algorithm groups similar data together based on selected features. For financial transactions, these features might include transaction amount, location, timing, and so on. By clustering data, investigators can identify patterns or trends that might indicate fraudulent behavior. For example, unusually large transactions or clusters of transactions happening at odd hours may stand out.

2. Anomaly Detection:

Clusters formed by the K-means algorithm can help identify what is "normal" within a dataset. Transactions that fall outside these normal clusters (i.e., anomalies) can be flagged for further investigation. These anomalies could potentially represent fraudulent transactions.

3. Simplifying Large Datasets:

Financial investigations often involve large datasets, which can be overwhelming. The K-means algorithm can help simplify these large datasets by grouping similar transactions together. Investigators can then analyze these clusters instead of having to go through each transaction individually.

4. Unsupervised Learning:

K-means is an unsupervised learning algorithm, which means it doesn't require pre-labeled data to function. This is beneficial in fraud detection, where new fraud techniques are continually evolving, and labeled datasets of all possible fraud types might not exist.

5. Detecting Evolving Fraud Techniques:

Fraud techniques often evolve to evade detection. By continuously applying the K-means algorithm to new data, it's possible to stay updated on emerging trends and patterns in financial transactions. This could potentially help detect new fraud techniques.

6.2 SWOT analysis

6.2.1 Strengths

Interactive Visualization: The application provides an interactive 2D graphical view, allowing the user to directly interact with the data points. The user can move data points, select multiple points by dragging, and view the details of a point by hovering over it.

Comments and Annotations: The application allows the user to add comments to individual data points and groups of data points, which can be very useful for data analysis and reporting.

PDF Report Generation: The application can generate a PDF report that includes a title, a main comment, and all the additional comments. This feature makes it easy to document and share the analysis results.

Multi-threading: The application uses multi-threading to generate the PDF report, which helps to keep the GUI responsive.

6.2.2 Weaknesses

Global Variables: The application uses a global variable `global-comments` to store group comments. Using global variables can make the code harder to understand and maintain, and it can lead to unexpected behavior if the variable is accidentally modified in different parts of the code.

Lack of Error Handling: The application doesn't seem to have much error handling. For example, it doesn't handle potential errors that could occur when reading the data file, or when the user tries to save the PDF file to a location where they don't have write permission.

User Interface: While the application provides many features, the user interface could be improved. For instance, it might not be immediately clear to a new user what each button does or how to use the drag-select mode.

Hardcoded Parameters: Some parameters are hardcoded, such as the colors in `DotColorManager` and the number of clusters in `load-dots` method. It might be more flexible to allow the user to specify these parameters.

Limited Scalability: The application seems to be designed for handling small to medium-sized datasets. For larger datasets, performance might be an issue, especially for the interactive features like moving and selecting data points.

6.2.3 Opportunities

Scalability Improvements: The application could be improved to handle larger datasets more efficiently, making it useful for more complex tasks. **Additional Features:** There is potential to add more features such as different types of plots, more customization options, better error handling, and a more intuitive user interface. **Integration with Other Tools:** The application could be integrated with other data analysis tools to provide more comprehensive functionalities.

6.2.4 Threats

Competition: There are many powerful and well-established data visualization and analysis tools available, which could make it hard for this application to stand out. **Technological Advances:** Rapid advances in technology and programming languages could make some aspects of the application outdated quickly. **User Expectations:** Users may have high expectations for functionality, ease of use, and performance, which could be challenging to meet.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

The application has proven easy to use and intuitive, while there are no tutorials or sections dedicated to them they should still be easy for most of the users to use. The GUI provides most of the information about the data points and the investigator has enough tools to conduct a investigation and correct something if some problems may arise.

7.2 Future Work

While the application can do digital proof abstraction for financial fraud dataset but in the future the app can be adapted into a fully fledged digital analysis app that can do any type of data abstraction. Several features can also be added like relationships and plotting individual files and letting the digital investigator edit and comment on them. When it comes to printing reports adding statistics and adapting the app for everything listed above would also be important.

Bibliography

- [1] Trevor Hastie, Robert Tibshirani, and Jerome Friedman. *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. Springer Science & Business Media, 2009.
- [2] Anil K Jain. Data clustering: 50 years beyond k-means. *Pattern recognition letters*, 31(8):651–666, 2010.
- [3] Felix Freiling, Thomas Glanzmann, and Hans P. Reiser. Characterizing loss of digital evidence due to abstraction layers. *Digital Investigation*, 20:S107–S115, 2017. DFRWS 2017 Europe.
- [4] Shahzad Saleem, Oliver Popov, and Ibrahim Bagilli. Extended abstract digital forensics model with preservation and protection as umbrella principles. *CORE*, 2014.
- [5] Sandeepak Bhandari and Vacius Jusas. An abstraction based approach for reconstruction of timeline in digital forensics. *Symmetry*, 2020.
- [6] Eoghan Casey. *Digital Evidence and Computer Crime: Forensic Science, Computers and the Internet*. Academic Press, 2011.
- [7] Wes McKinney. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython*. O’Reilly Media, Inc., 2013.
- [8] Mark Summerfield. *Rapid GUI Programming with Python and Qt: The Definitive Guide to PyQt Programming*. Pearson Education, 2007.
- [9] Pavel Berkhin. A survey of clustering data mining techniques. *Grouping multidimensional data*, pages 25–71, 2006.
- [10] Chandan Bajaj and Soyeb Nagori. K-means: A new hybrid feature selection method. *International Journal of Engineering and Technology*, 6(2):1118–1121, 2014.
- [11] John A Hartigan and Manchek A Wong. Algorithm as 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 28(1):100–108, 1979.

- [12] Joel Grus. *Data Science from Scratch: First Principles with Python*. O'Reilly Media, Inc., 2015.
- [13] Bela Frigyik, Amol Srivastava, and Maya R. Gupta. An introduction to kernel and nearest-neighbor nonparametric regression. 2008.
- [14] Guido van Rossum and Fred L. Drake. *Programming in Python 3: A Complete Introduction to the Python Language*. Addison-Wesley Professional, 2011.
- [15] S Brilliant. Cluster analysis and k-means clustering algorithm. 2007.
- [16] Mark Pilgrim. *Dive Into Python: Python from Novice to Pro*. Apress, 2004.
- [17] Paul Barry. *Head First Python: A Brain-Friendly Guide*. O'Reilly Media, Inc., 2010.
- [18] Flanagan David. Machine learning: The art and science of algorithms that make sense of data. *Nature*, 24(12):1897–1899, 2016.
- [19] Nathan Dunning and Eric Friedman. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications Co., 2014.
- [20] Ian H Witten, Eibe Frank, Mark A Hall, and Christopher J Pal. *Data Mining: Practical Machine Learning Tools and Techniques*. Morgan Kaufmann, 2011.
- [21] Richard S Sutton and Andrew G Barto. *Reinforcement Learning: An Introduction*. MIT press, 2018.